

MODUL PRAKTIKUM

Pertemuan 1

Hello World & Setup Flutter

Tools: Android Studio Panda + Flutter SDK 3.38.5

Durasi: 100 menit | Prasyarat: Dart Fundamental

1. Apa Itu Flutter?

Flutter adalah framework UI open-source dari Google yang memungkinkan developer membangun aplikasi untuk berbagai platform — Android, iOS, Web, Windows, macOS, dan Linux — hanya dari satu basis kode (single codebase) yang ditulis dalam bahasa Dart.

Flutter pertama kali diperkenalkan Google pada tahun 2017 dan resmi rilis versi stabil (1.0) pada Desember 2018. Sejak saat itu, Flutter berkembang pesat dan kini menjadi salah satu framework mobile development paling populer di dunia.

1.1 Kenapa Flutter di Era Sekarang?

Di era 2024–2025, kebutuhan bisnis bergerak lebih cepat dari sebelumnya. Perusahaan tidak lagi punya waktu dan budget untuk membangun dua aplikasi terpisah (satu untuk Android, satu untuk iOS). Flutter hadir sebagai solusi nyata untuk masalah ini.

Keunggulan	Penjelasan	Dampak Nyata
Single Codebase	Satu kode Dart menghasilkan APK Android, IPA iOS, Web, Desktop sekaligus.	Hemat waktu pengembangan hingga 50%
Hot Reload < 1 detik	Perubahan kode langsung tampil di emulator tanpa restart. State tetap terjaga.	Developer 3x lebih produktif saat UI iteration
Performa Native	Flutter compile ke kode ARM/x64 native, bukan WebView. Animasi 60–120 fps.	UX setara aplikasi native Android/iOS
Ekosistem Besar	pub.dev menyediakan 30.000+ package siap pakai (HTTP, Maps, Auth, dll).	Jarang perlu bangun fitur dari nol
Permintaan Industri Tinggi	Flutter masuk 3 besar framework mobile terpopuler (Stack Overflow Survey 2024).	Skill yang langsung relevan di dunia kerja

Flutter di Dunia Nyata

Aplikasi besar yang dibangun dengan Flutter: Google Pay, BMW App, eBay Motors, Alibaba Xianyu, Nubank (bank digital terbesar Amerika Latin).

Di Indonesia: GoTo, Tokopedia, dan banyak startup fintech lokal menggunakan Flutter untuk aplikasi mereka. Startup yang mengadopsi Flutter sejak awal bisa go-live di Android & iOS dalam waktu bersamaan — keunggulan kompetitif yang signifikan.

1.2 Flutter vs Java vs Kotlin — Apa Bedanya?

Sebagai praktikan yang sebelumnya belajar Java, wajar jika muncul pertanyaan: 'Kenapa tidak pakai Java atau Kotlin saja untuk bikin aplikasi Android?' Berikut perbandingan jujur nya:

Perbandingan Umum

Aspek	Java (Android)	Kotlin (Android)	Flutter (Dart)
Target Platform	Android saja	Android saja	Android, iOS, Web, Desktop — semua dari 1 kode
Bahasa	Java	Kotlin	Dart (mirip Java/JS, mudah dipelajari)
UI Framework	XML Layout + View System	XML Layout / Compose	Widget Tree (semua dalam kode Dart)
Hot Reload	Tidak ada	Terbatas (Compose Preview)	Ya — perubahan tampil < 1 detik
Performa	Native	Native	Near-native (compile ke ARM/x64)
Kurva Belajar	Sedang	Mudah (jika sudah Java)	Mudah (Dart mirip Java/JS)
Popularitas 2024	Menurun untuk mobile baru	Standar Android resmi Google	Top 3 framework mobile dunia

Perbandingan Sintaks — Terasa Familiar!

Kalau kamu sudah bisa Java, belajar Dart tidak akan terasa asing. Lihat perbandingan berikut:

Konsep	Java	Dart (Flutter)
Deklarasi variabel	<code>String nama = "Budi";</code>	<code>String nama = 'Budi';</code> atau <code>var nama = 'Budi';</code>
Fungsi / method	<code>void sapa(String n) { ... }</code>	<code>void sapa(String n) { ... } // identik!</code>
Class	<code>class Mobil { ... }</code>	<code>class Mobil { ... } // identik!</code>
Constructor	<code>public Mobil(String merk) { }</code>	<code>Mobil(this.merk); // lebih ringkas</code>
Kondisi if	<code>if (x > 0) { ... }</code>	<code>if (x > 0) { ... } // identik!</code>
Loop for	<code>for (int i=0; i<10; i++) { }</code>	<code>for (int i=0; i<10; i++) { } // identik!</code>
List / ArrayList	<code>ArrayList<String> list = new ArrayList<>();</code>	<code>List<String> list = []; // lebih ringkas</code>
Null safety	Tidak ada (NullPointerException!)	Built-in: <code>String?</code> bisa null, <code>String</code> tidak bisa

Kesimpulan: Java ke Flutter itu Mudah

Logika pemrograman (if, for, class, OOP) yang sudah kamu kuasai di Java langsung berlaku di Dart. Perbedaan terbesar bukan di bahasa, tapi di cara membangun UI: Flutter pakai Widget (kode), bukan XML layout seperti Android native.

Keuntungan belajar Flutter setelah Java: kamu sudah paham OOP, tinggal adaptasi ke Widget paradigm — biasanya 1-2 minggu sudah terasa nyaman.

Kotlin adalah 'Java yang lebih modern' untuk Android native. Flutter adalah 'lompatan lebih jauh' — satu kode untuk semua platform.

Kapan Pakai Apa?

Situasi	Rekomendasi	Alasan
Mau bikin app Android + iOS sekaligus	Flutter	Single codebase = hemat waktu & biaya
Sudah ada tim Android native yang established	Kotlin	Tidak perlu migrasi yang berisiko
Butuh akses fitur hardware sangat spesifik	Kotlin / Java	Native punya akses API paling lengkap
Startup / project baru dengan tim kecil	Flutter	Satu developer bisa cover semua platform
Kuliah / belajar pemrograman mobile pertama kali	Flutter	Hot reload mempercepat belajar eksperimen

2. Informasi Umum Praktikum

Pertemuan	Minggu 1 (Setup Awal)
Topik	Pengenalan Flutter, Instalasi Android Studio Panda, Hot Reload
Durasi	100 menit
Prasyarat	Dart Fundamental
Tools	Android Studio Panda + Flutter SDK 3.38.5

2.1 Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa mampu:

1. Memastikan Flutter SDK, Android Studio Panda, dan emulator siap pakai
2. Melakukan upgrade Android Studio (Otter → Panda) tanpa kehilangan settings
3. Membuat project Flutter pertama dari Android Studio GUI Wizard maupun CLI
4. Menjalankan aplikasi di emulator/device dan menggunakan Hot Reload
5. Membaca dan memahami struktur project Flutter (folder lib/, pubspec.yaml)
6. Mengubah aplikasi default menjadi Hello World versi sendiri
7. Mengenal widget dasar: MaterialApp, Scaffold, Center, Text, Container, Row, Column, Icon, SizedBox
8. Membedakan StatelessWidget dan flow build → render

2.2 Gambaran Hasil Akhir

Pada akhir praktikum ini, mahasiswa akan memiliki 2 project:

- pertemuan_1 — aplikasi Hello World dengan kartu profil sederhana
- latihan_widget — aplikasi tempat eksperimen 4 latihan widget dasar

3. Langkah 0 — Update Android Studio (Opsional, 10 menit)

Bila sudah punya Android Studio versi sebelumnya (Otter atau lebih lama), upgrade ke Panda tidak perlu uninstall terlebih dahulu.

Langkah-langkah:

9. Download installer Android Studio Panda dari developer.android.com/studio
10. Buka file .dmg, lalu drag icon Android Studio ke folder Applications
11. Akan muncul dialog konfirmasi — pilih Replace

Opsi	Efek
Replace (Direkomendasikan)	Versi lama ditimpa. Settings, plugin (Flutter & Dart), SDK, AVD tetap aman karena tersimpan di <code>~/Library/Application Support/Google/AndroidStudio*</code>
Keep Both	Dua versi coexist, boros disk ~3-4 GB. Hanya untuk testing.
Stop	Batalkan instalasi.

12. Buka Android Studio Panda → pilih Import Studio Settings → semua plugin & config termigrasi
13. Pastikan plugin Flutter dan Dart masih aktif: Preferences → Plugins → Installed

4. Langkah 1 — Verifikasi Environment (10 menit)

Sebelum menulis kode, pastikan komputer sudah siap. Buka terminal dan jalankan:

```
flutter --version
```

Output yang diharapkan:

```
Flutter 3.38.5 • channel stable • https://github.com/flutter/flutter.git
Tools • Dart 3.x.x
```

Cek path Flutter SDK:

```
which flutter
# /opt/homebrew/bin/flutter
readlink -f $(which flutter) | sed 's|/bin/flutter|'
# /opt/homebrew/share/flutter
```

Jalankan flutter doctor untuk memeriksa kelengkapan tooling:

```
flutter doctor
```

Pastikan minimal item berikut bertanda centang hijau:

Item	Status yang Dibutuhkan
Flutter (Channel stable)	✓ Centang hijau
Android toolchain	✓ Centang hijau
Android Studio (Panda)	✓ Centang hijau
Connected device	✓ Centang hijau (emulator atau HP fisik)

Jika Ada Error di Android toolchain:

Jalankan: flutter doctor --android-licenses

Ketik y untuk menerima semua lisensi yang muncul.

4.1 Siapkan Emulator atau Device

Opsi A — Emulator Android (Rekomendasi untuk Lab)

14. Buka Android Studio → menu Tools → Device Manager
15. Klik Create Virtual Device → pilih Pixel 7 atau Medium Phone → Next
16. Pilih image API 33 / 34 → Finish
17. Klik tombol ► untuk menjalankan emulator

Opsi B — HP Android Fisik

18. Aktifkan Developer Options: ketuk Build Number 7 kali di Settings → About Phone
19. Aktifkan USB Debugging
20. Hubungkan HP ke laptop dengan kabel USB
21. Tap Allow ketika dialog "Allow USB debugging" muncul

Verifikasi device terdeteksi:

```
flutter devices
# sdk gphone64 arm64 (mobile) • emulator-5554 • android-arm64 • Android 14
(API 34)
```

5. Langkah 2 — Membuat Project Flutter Pertama (15 menit)

Ada dua cara membuat project Flutter. Pilih salah satu sesuai preferensi.

5.1 Cara GUI — Lewat Android Studio Wizard (Direkomendasikan)

22. Buka wizard: File → New → New Project...
23. Di sidebar kiri, scroll ke bagian Generators → klik Flutter
24. Isi Flutter SDK path: /opt/homebrew/share/flutter
25. Klik Next, lalu isi form konfigurasi project:

Field	Nilai	Catatan
Project name	pertemuan_1	Wajib snake_case. Tidak boleh ada spasi, dash, atau huruf besar.
Project location	~/Documents/Projek/Praktikum/pertemuan_1	Lokasi penyimpanan project.
Project type	Application	Untuk app (bukan plugin/package).
Organization	com.example	Jadi package ID Android. Sulit diganti setelah project dibuat.
Android language	Kotlin	Standar modern.
Platforms	Android & iOS saja	Linux/MacOS/Web/Windows uncheck dulu.

26. Klik Create. Android Studio akan menjalankan flutter create + flutter pub get (1-2 menit).

5.2 Cara CLI — Lewat Terminal

```
cd ~/Documents/Projek/Praktikum
flutter create pertemuan_1
cd pertemuan_1
```

5.3 Jalankan Aplikasi Default

Dari Android Studio: klik tombol Run hijau atau tekan Ctrl+R (Mac) / Shift+F10 (Windows/Linux).

Dari terminal:

```
flutter run
```

Tunggu proses build (~1-3 menit pada run pertama). Setelah selesai, muncul aplikasi Counter bawaan Flutter dengan tombol + di kanan bawah.

5.4 Coba Hot Reload (PENTING!)

Kenapa Hot Reload Penting?

Hot Reload adalah fitur yang membuat Flutter jauh lebih menyenangkan dibanding Android native.

Di Android native (Java/Kotlin), setiap kali ubah XML layout, kamu harus build ulang (~30 detik–2 menit).

Di Flutter, perubahan tampil dalam < 1 detik tanpa kehilangan state aplikasi. Ini mengubah cara kita belajar UI!

Tanpa menutup aplikasi, buka file lib/main.dart. Cari baris:

```
title: 'Flutter Demo Home Page',
```

Ubah menjadi:

```
title: 'Coba Hot Reload!',
```

Simpan file (Cmd+S / Ctrl+S) — di Android Studio, save = otomatis hot reload.

Tombol Terminal	Shortcut Android Studio	Fungsi
r	Cmd+S (auto saat save)	Hot Reload — refresh kode tanpa kehilangan state
R	Cmd+Shift+\	Hot Restart — restart penuh, state direset
q	Cmd+F2	Quit — hentikan aplikasi

Aturan Tak Tertulis Flutter:

Selama ngoding Flutter, jangan pernah hentikan run process. Semua perubahan UI cukup save → hot reload.

6. Langkah 3 — Memahami Struktur Project (10 menit)

Buka folder pertemuan_1. Inilah struktur yang akan kamu temui:

```
pertemuan_1/
  lib/           ← TEMPAT KITA NGODING (99% kerja di sini)
    main.dart    ← entry point aplikasi
  android/       ← konfigurasi Android (jarang disentuh)
  ios/           ← konfigurasi iOS (jarang disentuh)
  test/          ← unit test
  pubspec.yaml   ← daftar dependency & metadata project
  pubspec.lock   ← versi dependency yang ter-resolve (auto-generated)
```

6.1 Bedah lib/main.dart

Buka lib/main.dart. Kode ~120 baris. Mari bedah bagian utamanya:

```
import 'package:flutter/material.dart'; // 1. Import library Material Design

void main() {                                // 2. Entry point aplikasi
  runApp(const MyApp());                    // runApp() = 'tampilkan widget
  ini'
}

class MyApp extends StatelessWidget {         // 3. Root widget aplikasi
  @override
  Widget build(BuildContext context) {       // 4. build() = mengembalikan
  tampilan
    return MaterialApp(
      title: 'Flutter Demo',
      home: const MyHomePage(...),         // 5. Halaman pertama yang
  ditampilkan
    );
  }
}
```

6.2 Konsep Kunci

Konsep	Penjelasan	Analogi dari Java
Widget	Semua di Flutter adalah widget — teks, tombol, padding, bahkan tema	Seperti View/ViewGroup di Android
build() method	Method yang mengembalikan widget tree — dipanggil setiap kali UI perlu di-render	Seperti onDraw() tapi lebih tinggi level
StatelessWidget	Widget yang tidak berubah setelah dibangun (statis)	Seperti class biasa tanpa mutable state
StatefulWidget	Widget yang bisa berubah (dipelajari praktikum berikutnya)	Seperti class dengan setter + notifikasi

BuildContext	'Lokasi' widget di pohon — dipakai untuk akses tema, navigasi, dll	Seperti Context di Android Activity
--------------	--	-------------------------------------

6.3 Flow Build → Render (StatelessWidget)

Apa yang sebenarnya terjadi ketika runApp() dipanggil? Inilah alurnya:

Tahap	Yang Terjadi
1. main()	Fungsi pertama yang dipanggil Dart saat aplikasi dibuka
2. runApp(MyApp())	Flutter diberi widget root untuk ditampilkan
3. build() dipanggil	MyApp.build() return MaterialApp → build rekursif ke bawah
4. Widget Tree terbentuk	Struktur logis: 'apa yang harus tampil'
5. Element Tree dibuat	Instans aktual yang dipasang di memori
6. Render Tree menggambar	Engine Skia/Impeller paint ke layar (60 fps)

Aturan Emas build():

Method build() bisa dipanggil berkali-kali oleh Flutter (misal saat orientasi layar berubah).
Jangan lakukan operasi mahal di dalamnya — cukup susun widget dan return.

6.4 Bedah pubspec.yaml

File ini = package.json di JavaScript atau requirements.txt di Python. Semua dependency project ditulis di sini.

```
name: pertemuan_1
description: A new Flutter project.
version: 1.0.0+1

environment:
  sdk: ^3.0.0          # versi Dart yang dibutuhkan

dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.6 # icon style iOS
```

Setiap kali butuh package baru (misal http untuk API), tambahkan di sini lalu jalankan:

```
flutter pub get
```

7. Langkah 4 — Hello World Versi Sendiri (30 menit)

Ganti seluruh isi lib/main.dart dengan versi minimal kita sendiri, lalu kembangkan bertahap.

7.1 Versi Termurni: Teks di Tengah Layar

Ganti seluruh isi lib/main.dart dengan kode berikut, lalu tekan Hot Restart:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        body: Center(
          child: Text('Halo, Flutter!'),
        ),
      ),
    );
  }
}
```

Hasil:

Layar putih dengan tulisan 'Halo, Flutter!' di tengah.

7.2 Tambah AppBar

```
home: Scaffold(
  appBar: AppBar(
    title: const Text('Hello Flutter'),
    backgroundColor: Colors.blue,
    foregroundColor: Colors.white,
  ),
  body: const Center(
    child: Text('Halo, Flutter!'),
  ),
),
```

Save (hot reload). Sekarang ada bar biru di atas dengan judul 'Hello Flutter'.

7.3 Styling Teks

```
const Text(  
  'Halo, Flutter!',  
  style: TextStyle(  
    fontSize: 32,  
    fontWeight: FontWeight.bold,  
    color: Colors.blue,  
  ),  
)
```

7.4 Layout Vertikal dengan Column

```
body: Center(  
  child: Column(  
    mainAxisAlignment: MainAxisAlignment.center,  
    children: [  
      const Text('\u{1F44B}', style: TextStyle(fontSize: 64)),  
      const SizedBox(height: 16),  
      const Text(  
        'Halo, [Nama Anda]!',  
        style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold),  
      ),  
      const SizedBox(height: 8),  
      const Text(  
        'Selamat datang di dunia Flutter.',  
        style: TextStyle(fontSize: 14, color: Colors.grey),  
      ),  
    ],  
  ),  
)
```

Ganti [Nama Anda] dengan nama kamu sebenarnya!

7.5 Kartu Profil dengan Container

Setelah `SizedBox(height: 8)` terakhir, tambahkan:

```
const SizedBox(height: 24),  
Container(  
  margin: const EdgeInsets.symmetric(horizontal: 32),  
  padding: const EdgeInsets.all(20),  
  decoration: BoxDecoration(  
    color: Colors.blue.shade50,  
    borderRadius: BorderRadius.circular(16),  
    border: Border.all(color: Colors.blue.shade200),  
  ),  
)
```

```

),
child: const Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Text('NIM: 123456789', style: TextStyle(fontSize: 16)),
    SizedBox(height: 4),
    Text('Prodi: Teknik Informatika', style: TextStyle(fontSize: 16)),
    SizedBox(height: 4),
    Text('Semester: 5', style: TextStyle(fontSize: 16)),
  ],
),
),
),

```

Ganti NIM, Prodi, dan Semester sesuai data kamu.

7.6 Tambah Tombol

```

const SizedBox(height: 24),
ElevatedButton(
  onPressed: () {
    // Belum dipakai – akan dipelajari di praktikum berikutnya
  },
  child: const Text('Tap Saya'),
),

```

Selamat!

Kamu baru saja membangun Hello World versi sendiri lengkap dengan kartu profil!

Diagram Widget Tree Hasil Akhir

```

MaterialApp
  Scaffold
    AppBar
      Text ('Hello Flutter')
    body: Center
      Column
        Text (emoji)
        SizedBox
        Text ('Halo, [Nama]!')
        SizedBox
        Text ('Selamat datang...')
        SizedBox
        Container ← kartu profil biru
          Column
            Text (NIM)
            Text (Prodi)
            Text (Semester)
          SizedBox
        ElevatedButton

```

Text ('Tap Saya')

8. Langkah 5 — Latihan Widget Dasar (40 menit)

Hentikan run process, lalu buat project kedua untuk eksperimen widget. Project ini terpisah agar kode Hello World tetap rapi.

```
cd ~/Documents/Projek/Praktikum
flutter create latihan_widget
cd latihan_widget
flutter run
```

Untuk setiap latihan, ganti seluruh isi lib/main.dart dengan kode yang diberikan, hot restart, amati hasilnya, lalu eksperimen sesuai instruksi.

Latihan 1: Text & Styling

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MaterialApp(
    debugShowCheckedModeBanner: false,
    home: Scaffold(
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text('Hello Flutter!',
              style: TextStyle(
                fontSize: 28, fontWeight: FontWeight.bold, color:
Colors.blue),
            ),
            SizedBox(height: 8),
            Text('Ini teks biasa dengan ukuran kecil',
              style: TextStyle(fontSize: 14, color: Colors.grey),
            ),
          ],
        ),
      ),
    ),
  ));
}
```

Eksperimen yang harus dicoba:

- Ubah fontSize ke 40, lalu ke 12 — apa yang terjadi?
- Ganti FontWeight.bold ke FontWeight.w300 (tipis), w900 (sangat tebal)
- Ganti Colors.blue ke Colors.red, Colors.deepPurple, Color(0xFF2196F3)
- Tambahkan property letterSpacing: 2 di TextStyle pertama

Latihan 2: Container & Decoration

```
import 'package:flutter/material.dart';

void main() {
  runApp(MaterialApp(
    debugShowCheckedModeBanner: false,
    home: Scaffold(
      body: Center(
        child: Container(
          width: 200, height: 200,
          padding: const EdgeInsets.all(20),
          decoration: BoxDecoration(
            color: Colors.blue,
            borderRadius: BorderRadius.circular(20),
            boxShadow: [
              BoxShadow(
                color: Colors.blue.withValues(alpha: 0.3),
                blurRadius: 20,
                offset: const Offset(0, 10),
              ),
            ],
          ),
          child: const Center(
            child: Text('Box',
              style: TextStyle(color: Colors.white, fontSize: 24)),
          ),
        ),
      ),
    ),
  ));
}
```

Eksperimen yang harus dicoba:

- Ubah width dan height ke nilai berbeda (misal 300x100)
- Ganti `BorderRadius.circular(20)` → `BorderRadius.circular(100)` — bentuk apa yang muncul?
- Ubah `blurRadius: 20` ke `blurRadius: 50` — perhatikan bayangannya
- Tambahkan border: `Border.all(color: Colors.black, width: 4)` di dalam `BoxDecoration`

Insight:

Container = 'kotak ajaib' Flutter. Hampir semua kartu, tombol custom, dan bingkai dibuat dari Container.

Latihan 3: Row & Column

Kode lengkap tersedia di modul digital. Eksperimen yang harus dicoba (`MainAxisAlignment` di Row):

Nilai	Pengaruh
.start	Widget tumpuk di kiri
.center	Widget tumpuk di tengah
.end	Widget tumpuk di kanan
.spaceBetween	Pertama di kiri, terakhir di kanan, sisa di tengah
.spaceAround	Jarak di tepi = $\frac{1}{2}$ jarak antar widget
.spaceEvenly	Semua jarak sama rata

Bonus: ada `crossAxisAlignment` untuk sumbu tegak lurus. Coba tambahkan di Row dan ubah nilainya.

Latihan 4: Icon & Bottom Bar Mock-up

Eksperimen yang harus dicoba:

- Buka <https://fonts.google.com/icons> — pilih 4 ikon yang kamu suka, ganti Icons.home, Icons.receipt_long, dst.
- Ubah size: 32 ke ukuran berbeda (24, 48, 64)
- Ganti warna ikon pertama merah, kedua hijau, ketiga ungu

9. Ringkasan Widget Dasar

Widget	Fungsi	Analogi (Java/Android)
MaterialApp	Bungkus aplikasi (tema, navigasi)	Application class
Scaffold	Kerangka halaman (AppBar, body, FAB)	Activity layout template
AppBar	Bar judul di atas	ActionBar / Toolbar
Center	Posisikan child di tengah layar	gravity='center'
Column	Susun widget secara vertikal	LinearLayout (vertical)
Row	Susun widget secara horizontal	LinearLayout (horizontal)
Container	Kotak dengan dekorasi (warna, border, shadow)	View + background + padding
Text	Menampilkan teks	TextView
Icon	Menampilkan ikon Material	ImageView dengan icon resource
SizedBox	Jarak/spacer antar widget	Space atau margin
ElevatedButton	Tombol terangkat	Button

10. Checklist Sebelum Pulang

- ☐ Android Studio Panda berhasil terinstall (atau upgrade dari Otter)
- ☐ flutter doctor tidak menunjukkan error fatal
- ☐ Emulator atau HP fisik berhasil terdeteksi via flutter devices
- ☐ Project pertemuan_1 berjalan dan menampilkan kartu profil dengan data Anda
- ☐ Hot reload terbukti bekerja — perubahan teks tampil instan
- ☐ Hot restart terbukti bekerja
- ☐ 4 latihan widget di project latihan_widget sudah dicoba (minimal 1 eksperimen per latihan)

11. Troubleshooting Umum

Masalah	Solusi
flutter: command not found	Tambahkan path Flutter SDK ke PATH di .zshrc/.bashrc
Field 'Flutter SDK path' kosong di wizard	Isi /opt/homebrew/share/flutter (Homebrew) atau path manual install Anda
Plugin Flutter tidak muncul setelah upgrade Studio	Preferences → Plugins → Marketplace → install ulang Flutter & Dart
No connected devices	Jalankan emulator dulu, atau colok HP & enable USB Debugging
Gradle build failed	Jalankan flutter clean lalu flutter run ulang
Unsupported class file major version XX	JDK terlalu baru/lama. Set JDK 17 di Preferences → Build → Gradle → Gradle JDK
Hot reload tidak bekerja setelah ubah class	Pakai Hot Restart (Cmd+Shift+\ / R) bukan Hot Reload
Layar putih saja, tidak ada apa-apa	Cek terminal/Run console — biasanya ada exception merah, baca pesannya
Target device masih macOS (desktop)	Klik dropdown device di toolbar atas, ganti ke emulator Android

Lampiran A — Cheatsheet Shortcut Android Studio

Aksi	macOS	Windows/Linux
Run	Ctrl + R	Shift + F10
Hot Reload	Cmd + S (auto saat save)	Ctrl + S
Hot Restart	Cmd + Shift + \	Ctrl + Shift + \
Stop Run	Cmd + F2	Ctrl + F2
Format kode	Cmd + Option + L	Ctrl + Alt + L
Cari file	Cmd + Shift + O	Ctrl + Shift + N
Cari simbol/class	Cmd + Option + O	Ctrl + Alt + Shift + N
Cari di semua tempat	Shift + Shift (double shift)	Shift + Shift
Auto-import	Option + Enter	Alt + Enter
Wrap widget (Wrap with...)	Option + Enter di nama widget	Alt + Enter
Rename simbol	Shift + F6	Shift + F6
Pindah baris ke atas/bawah	Cmd + Shift + ↑/↓	Ctrl + Shift + ↑/↓
Duplikasi baris	Cmd + D	Ctrl + D
Komentar baris	Cmd + /	Ctrl + /

Pro Tip:

Option + Enter (Mac) di nama widget → 'Wrap with Center / Padding / Column / Container' — fitur paling sering dipakai saat menyusun layout.

Lampiran B — Update Gradle & AGP

Project Flutter lama kadang error karena versi Gradle / Android Gradle Plugin (AGP) di bawah minimum Flutter terbaru.

Versi yang Aman Dipakai (per April 2026)

Komponen	Versi Minimum	Versi Direkomendasikan
Gradle	8.7	8.9
AGP (Android Gradle Plugin)	8.1.1	8.3.0
Kotlin	1.8.22	1.9.22
JDK	17	17

Langkah Perbaikan

27. Edit android/settings.gradle — naikan versi AGP:

```
id "com.android.application" version "8.3.0" apply false
id "org.jetbrains.kotlin.android" version "1.9.22" apply false
```

28. Edit android/gradle/wrapper/gradle-wrapper.properties — naikan versi Gradle:

```
distributionUrl=https\://services.gradle.org/distributions/gradle-8.9-all.zip
```

29. Bersihkan dan run ulang:

```
flutter clean
flutter pub get
flutter run
```

Catatan:

Run pertama setelah ganti versi akan men-download Gradle distribusi baru (~150 MB). Harap bersabar.

Referensi

- Flutter Official Docs: <https://docs.flutter.dev>
- Dart Language Tour: <https://dart.dev/language>
- Material 3 Design: <https://m3.material.io>
- DartPad (online editor): <https://dartpad.dev>
- Material Icons Catalog: <https://fonts.google.com/icons>

Preview Praktikum 2 (Minggu Depan)

Minggu depan kita mulai membangun aplikasi sungguhan. Skill yang baru saja kamu kuasai (Scaffold, Column, Row, Container, Text, Icon) akan langsung dipakai untuk membangun halaman Dashboard lengkap dengan kartu informasi dan daftar item.

Jangan hapus project pertemuan_1 dan latihan_widget — keduanya jadi referensi pribadi kamu!